

```

#
=====
=====
# CELL 0 PONDASI: RESEP ASLI TJ00LS (ANTI GESER)
#
=====
=====
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler,
MinMaxScaler
from sklearn.metrics import f1_score
from sklearn.model_selection import cross_val_score

print("\n Menyiapkan pondasi data dari awal (Menggunakan Resep
Asli)...")
file_bersih = "/Users/gitajilanmahira/Downloads/Gita_Jilan_Rifaskes
2/01b_Siap_MICE.xlsx"
df_ml = pd.read_excel(file_bersih)

y = df_ml['Y_Final_PKM']

# Gembok pengaman
daftar_hitam = ['Y_Final_PKM', 'Total_Kelompok_Obat_PKM', 'ID_cases',
'B1R1', 'B1R2']
for col in df_ml.columns:
    if col.startswith('Y_') or 'B1281' in col or 'B66' in col:
        if col not in daftar_hitam:
            daftar_hitam.append(col)

X = df_ml.drop(columns=[c for c in daftar_hitam if c in
df_ml.columns])

# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

#
-----
-----
# KERANJANG VARIABEL ASLI (TIDAK DIUBAH-UBAH LAGI!)
#
-----
-----

kolom_passthrough =
['B3R3', 'B1282R6', 'B14R15BTERSEDIA', 'B121R1', 'B121R7', 'B1282R1', 'B1282

```

```

R3', 'B122R11',

'B51R2DK2', 'B3R22E', 'B1282R5', 'B3R40', 'B126R4A', 'X_Ketersediaan_Dinkes',
,
'B3R10', 'B3R9B', 'B14R2',
'B8R2K4N', 'B4R10B', 'B121R5', 'B121R10', 'X_Persentase_Pemenuhan_EPurchasing',

'X_Sumber_Dana_Umum', 'X_Kondisi_Ruang_Farmasi', 'X_Kualitas_Gudang', 'X_Proporsi_Kapitasi', 'B4R3', 'B3R24', 'B14R16',
'X_Intensitas_Kapitasi', ]
kolom_nominal = [
'B3R6',
'X_Pengiriman_RK0', 'B121R4', 'B122R3', 'X_Metode_Pembelian_Kapitasi',
'X_Sumber_Dana_BPJS', 'B122R1',
'B51R1B', 'X_Kerusakan_Obat', 'B14R7A',
'X_Kepemilikan_Kendaraan', 'X_Sistem_Pencatatan',
]

kolom_numerik = [
'B3R11E', 'B3R13A', 'B4R14', 'B4R16A', 'B8R2K3N', 'B1282R2',
'B1282R4', 'B9R13A', 'B14R23'
]

kolom_nom_ada = [col for col in kolom_nominal if col in
X_train.columns]
kolom_num_ada = [col for col in kolom_numerik if col in
X_train.columns]

# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ('jalur_nom', OneHotEncoder(handle_unknown='ignore'),
kolom_nom_ada),
        ('jalur_num', StandardScaler(), kolom_num_ada)
    ], remainder='passthrough'
)

print("□ Pondasi SIAP! Kolom dijamin 100% sama dengan sejarah masa lalu.")

□ Menyiapkan pondasi data dari awal (Menggunakan Resep Asli)...
□ Pondasi SIAP! Kolom dijamin 100% sama dengan sejarah masa lalu.

from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

model_final = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(
        random_state=42,

```

```

        n_jobs=-1,
        n_estimators=500,
        max_depth=30,
        min_samples_split=2,
        min_samples_leaf=1,
        max_features='log2',
        bootstrap=False
    ))
])

model_final.fit(X_train, y_train)

nama_model = "Random Forest Tuned"

print("□ MODEL FINAL DIPILIH:", nama_model)
□ MODEL FINAL DIPILIH: Random Forest Tuned

nama_model = "Random Forest Tuned"

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Ambil model dan preprocessor dari pipeline
rf_model = model_final.named_steps['model']
prep = model_final.named_steps['prep']

# Nama fitur setelah preprocessing
nama_fitur = prep.get_feature_names_out()

# Feature importance bawaan Random Forest
importance = rf_model.feature_importances_

df_feature_importance = pd.DataFrame({
    'Fitur': nama_fitur,
    'Importance': importance
}).sort_values(by='Importance', ascending=False)

print("20 Feature Importance Teratas:")
print(df_feature_importance.head(20).to_string(index=False))

# Plot
top_n = 20

plt.figure(figsize=(10, 8))
plt.barh(
    df_feature_importance.head(top_n)['Fitur'][:-1],
    df_feature_importance.head(top_n)['Importance'][:-1]
)

```

```
plt.title('Top 20 Global Feature Importance - Random Forest Tuned',
fontsize=14, fontweight='bold')
plt.xlabel('Importance')
plt.ylabel('Fitur')
plt.tight_layout()
```

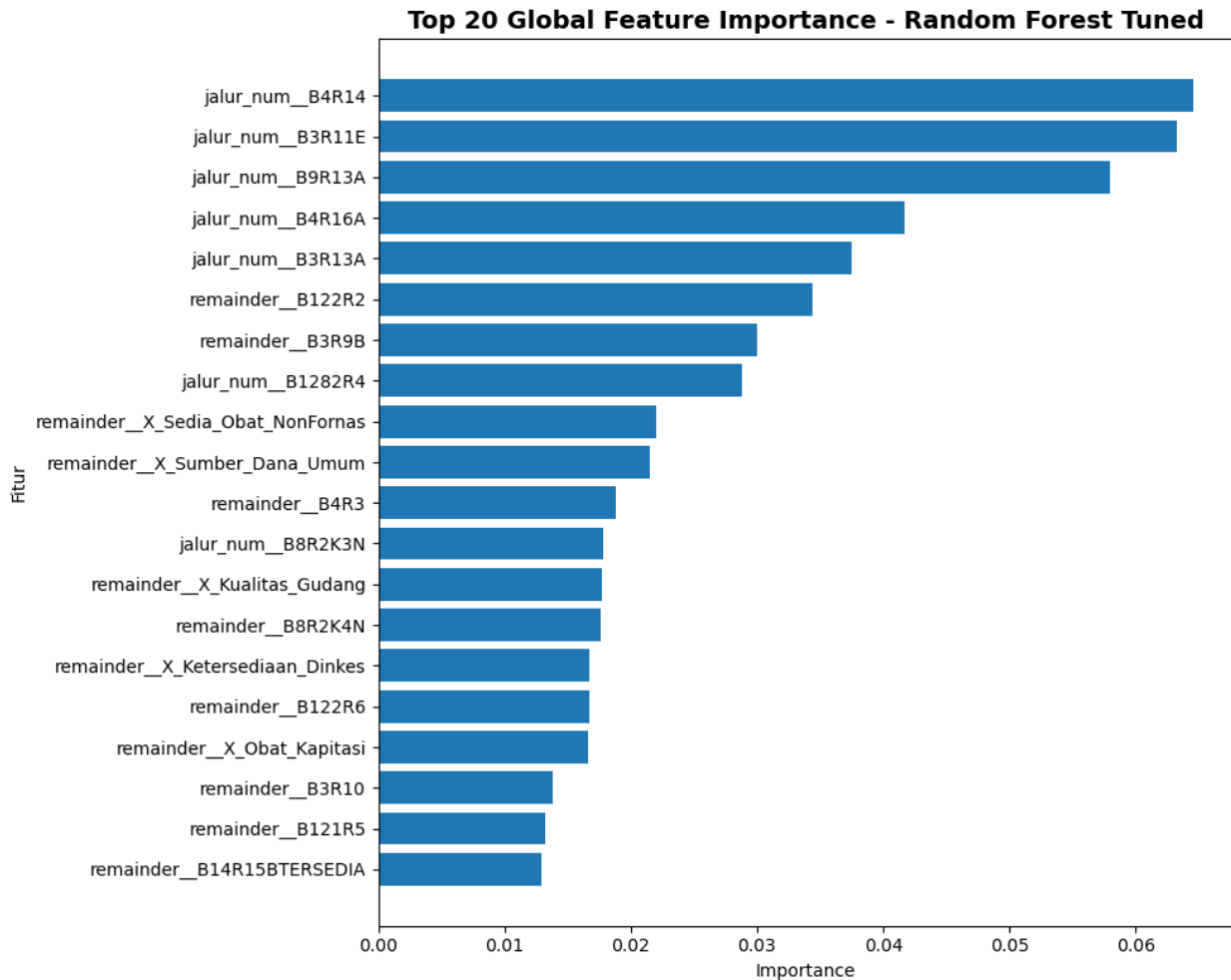
```
plt.savefig("/Users/gitajilanmahira/Downloads/Gita_Jilan_Rifaskes
2/Feature_Importance_Global_RF.png", dpi=300, bbox_inches='tight')
plt.show()
```

Export tabel

```
df_feature_importance.to_excel(
"/Users/gitajilanmahira/Downloads/Gita_Jilan_Rifaskes
2/Feature_Importance_Global_RF.xlsx",
index=False
)
```

20 Feature Importance Teratas:

	Fitur	Importance
	jalur_num__B4R14	0.064535
	jalur_num__B3R11E	0.063220
	jalur_num__B9R13A	0.058009
	jalur_num__B4R16A	0.041659
	jalur_num__B3R13A	0.037525
	remainder__B122R2	0.034342
	remainder__B3R9B	0.030031
	jalur_num__B1282R4	0.028780
remainder__X_Sedia_Obat_NonFornas		0.021987
remainder__X_Sumber_Dana_Umum		0.021536
	remainder__B4R3	0.018770
	jalur_num__B8R2K3N	0.017794
remainder__X_Kualitas_Gudang		0.017723
	remainder__B8R2K4N	0.017590
remainder__X_Ketersediaan_Dinkes		0.016755
	remainder__B122R6	0.016712
remainder__X_Obat_Kapitasi		0.016633
	remainder__B3R10	0.013779
	remainder__B121R5	0.013184
	remainder__B14R15BTERSEDIA	0.012931



```
!pip install shap
```

```
Requirement already satisfied: shap in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (0.51.0)
Requirement already satisfied: numpy>=2 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(2.3.4)
Requirement already satisfied: scipy in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(1.16.3)
Requirement already satisfied: scikit-learn in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(1.7.2)
Requirement already satisfied: pandas in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(2.3.3)
Requirement already satisfied: tqdm>=4.27.0 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(4.67.3)
Requirement already satisfied: packaging>20.9 in
```

```

/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(25.0)
Requirement already satisfied: slicer==0.0.8 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(0.0.8)
Requirement already satisfied: numba in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(0.65.0)
Requirement already satisfied: llvmlite in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(0.47.0)
Requirement already satisfied: cloudpickle in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(3.1.2)
Requirement already satisfied: typing-extensions in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from shap)
(4.15.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from pandas-
>shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from pandas-
>shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from pandas-
>shap) (2025.2)
Requirement already satisfied: six>=1.5 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from python-
dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.2.0 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from scikit-
learn->shap) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/opt/miniconda3/envs/uji/lib/python3.12/site-packages (from scikit-
learn->shap) (3.6.0)

import shap
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import sparse

# Transform X_test
X_test_transformed = model_final.named_steps['prep'].transform(X_test)

# Kalau hasilnya sparse, ubah jadi dense
if sparse.issparse(X_test_transformed):
    X_test_transformed = X_test_transformed.toarray()

# Buat dataframe SHAP

```

```

X_test_shap = pd.DataFrame(
    X_test_transformed,
    columns=model_final.named_steps['prep'].get_feature_names_out(),
    index=X_test.index
)

# Ambil model RF
rf_model = model_final.named_steps['model']

# TreeSHAP
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test_shap)

# Ambil SHAP untuk kelas 1
if isinstance(shap_values, list):
    shap_values_class1 = shap_values[1]
elif len(np.array(shap_values).shape) == 3:
    shap_values_class1 = shap_values[:, :, 1]
else:
    shap_values_class1 = shap_values

# SHAP bar plot global
shap.summary_plot(
    shap_values_class1,
    X_test_shap,
    plot_type="bar",
    max_display=20,
    show=False
)

plt.title("Global SHAP Feature Importance - Random Forest Tuned",
    fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig("/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes
2/SHAP_Global_Bar_RF.png", dpi=300, bbox_inches='tight')
plt.show()

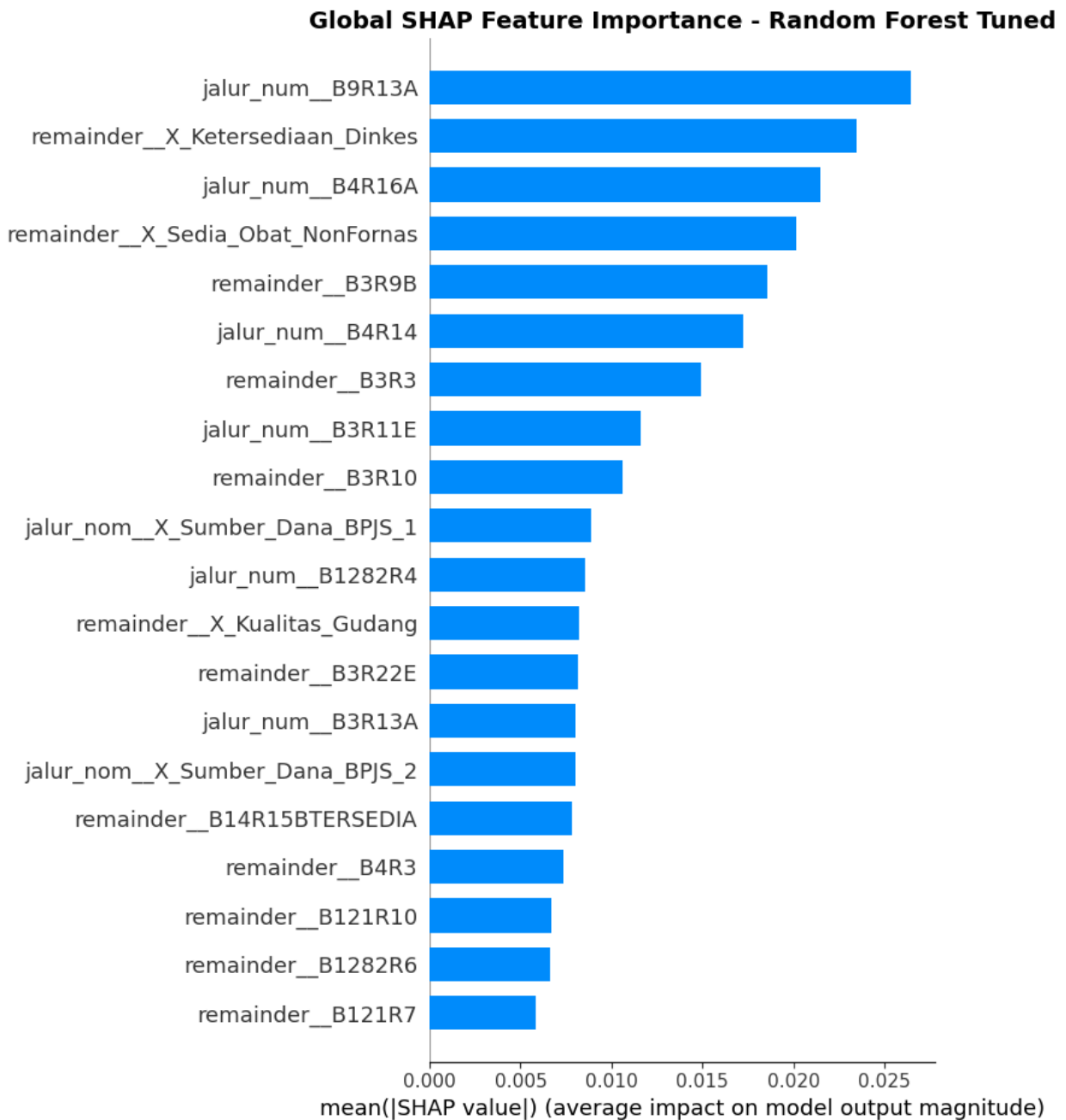
# SHAP beeswarm global
shap.summary_plot(
    shap_values_class1,
    X_test_shap,
    max_display=20,
    show=False
)

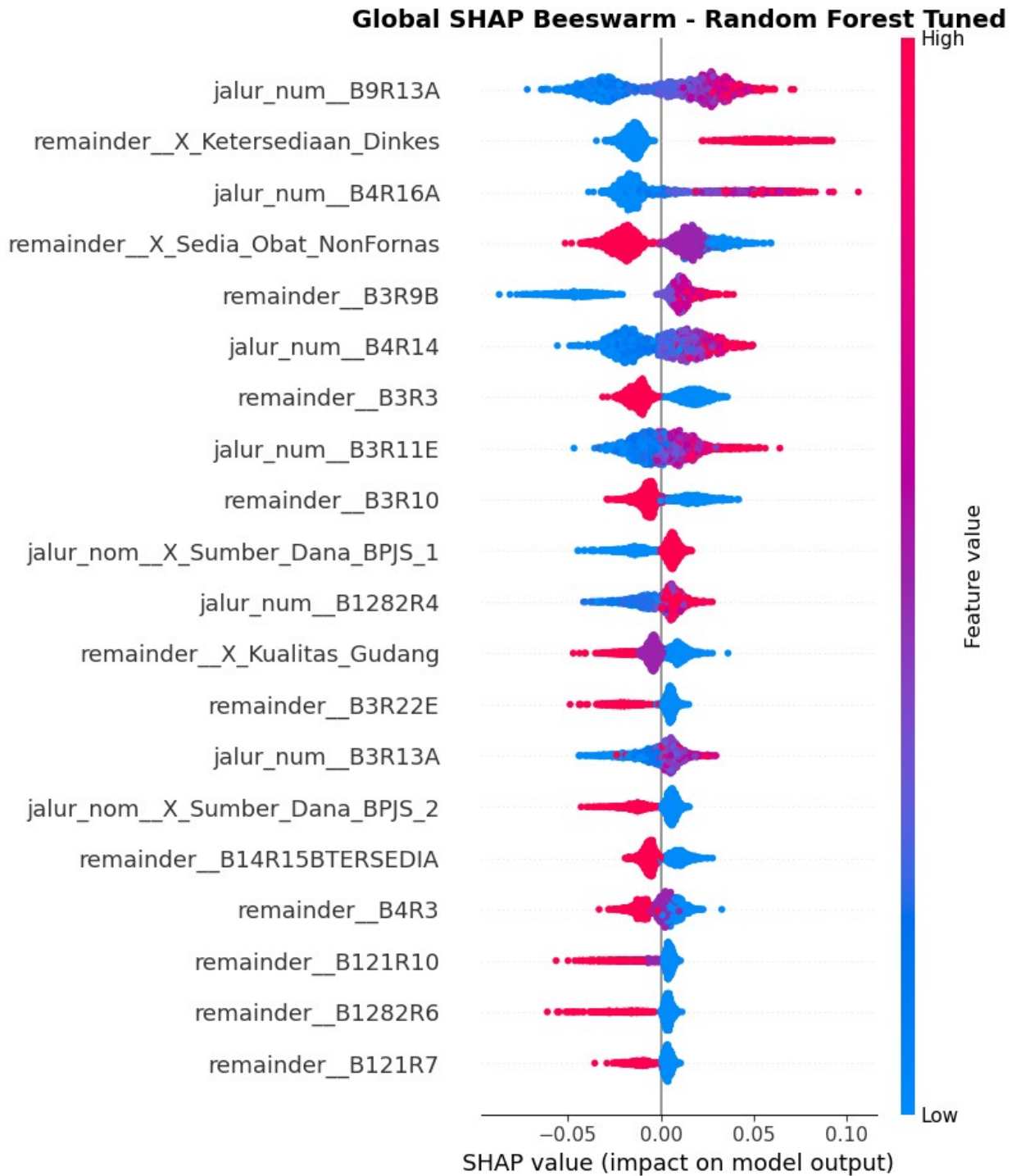
plt.title("Global SHAP Beeswarm - Random Forest Tuned", fontsize=14,
    fontweight='bold')
plt.tight_layout()
plt.savefig("/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes

```

```
2/SHAP_Global_Beeswarm_RF.png", dpi=300, bbox_inches='tight')  
plt.show()
```

```
/opt/miniconda3/envs/uji/lib/python3.12/site-packages/tqdm/auto.py:21:  
TqdmWarning: IProgress not found. Please update jupyter and  
ipywidgets. See  
https://ipywidgets.readthedocs.io/en/stable/user\_install.html  
from .autonotebook import tqdm as notebook_tqdm
```





```
# =====
# FIX TANPA ID_cases: AMBIL B1R1 & B3R6 BERDASARKAN INDEX X_test
# LALU METRIK + CONFUSION MATRIX B3R6 DAN JAWA-BALI
# =====

import os
```

```

import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import (
    accuracy_score, f1_score, matthews_corrcoef,
    average_precision_score, roc_auc_score, confusion_matrix
)

folder_path = "/Users/gitajilanmahira/Downloads/Gita Jilan Rifaskes 2"
output_dir = os.path.join(folder_path, "Analisis_Model_Final_RF")
os.makedirs(output_dir, exist_ok=True)

# df_raw harus sudah dibaca dari cell sebelumnya.
# Kalau belum ada, kasih tahu gue.
print("Cek df_raw ada?", "df_raw" in globals())

if "df_raw" not in globals():
    raise ValueError("df_raw belum ada. Run cell yang load file
01_Rifaskes_PKM dulu.")

print("Index X_test contoh:", X_test.index[:10].tolist())
print("Index df_raw range:", df_raw.index.min(), "sampai",
df_raw.index.max())

# Ambil metadata dari df_raw berdasarkan index X_test
try:
    meta_test = df_raw.loc[X_test.index, ["B1R1", "B3R6"]].copy()
    meta_test.index = X_test.index
    print("\n meta_test berhasil dibuat dari index X_test.")
    print(meta_test.head())
    print("\nMissing metadata:")
    print(meta_test[["B1R1", "B3R6"]].isna().sum())
except Exception as e:
    raise ValueError(
        "Index X_test tidak cocok dengan df_raw. Kalau ini gagal,
satu-satunya cara rapi adalah ulang train_test_split sambil membawa
meta_test."
    ) from e

# y_test aman
if not isinstance(y_test, pd.Series):
    y_test_series = pd.Series(y_test, index=X_test.index)
else:
    y_test_series = y_test.copy()

def ambil_kode_angka(x):
    if pd.isna(x):

```

```

        return np.nan
    teks = str(x).strip()
    angka = re.findall(r"\d+", teks)
    if len(angka) == 0:
        return np.nan
    return int(angka[0])

def safe_name(x):
    return str(x).replace("/", "_").replace(" ", "_").replace("-", "_")

def evaluasi_per_grup(model, X_data, y_data, grup_series, nama_kolom):
    hasil = []
    grup_series = grup_series.loc[X_data.index]
    y_data = y_data.loc[X_data.index]

    for grup in grup_series.dropna().unique():
        mask = grup_series == grup

        X_sub = X_data.loc[mask]
        y_sub = y_data.loc[mask]

        y_pred = model.predict(X_sub)
        y_proba = model.predict_proba(X_sub)[:, 1]

        try:
            auroc = roc_auc_score(y_sub, y_proba)
        except ValueError:
            auroc = np.nan

        try:
            aucprc = average_precision_score(y_sub, y_proba)
        except ValueError:
            aucprc = np.nan

        cm = confusion_matrix(y_sub, y_pred, labels=[0, 1])

        hasil.append({
            nama_kolom: grup,
            "Jumlah Data": len(y_sub),
            "Kelas 0": int(sum(y_sub == 0)),
            "Kelas 1": int(sum(y_sub == 1)),
            "Accuracy": accuracy_score(y_sub, y_pred),
            "F1 Score": f1_score(y_sub, y_pred, zero_division=0),
            "MCC": matthews_corrcoef(y_sub, y_pred),
            "AUCPRC": aucprc,
            "AUROC": auroc,
            "TN": cm[0, 0],
            "FP": cm[0, 1],
            "FN": cm[1, 0],
        })

```

```

        "TP": cm[1, 1]
    })

    return pd.DataFrame(hasil)

def plot_cm_per_grup(X_data, y_data, grup_series, prefix_file,
judul_prefix):
    grup_series = grup_series.loc[X_data.index]
    y_data = y_data.loc[X_data.index]

    for grup in grup_series.dropna().unique():
        mask = grup_series == grup

        X_sub = X_data.loc[mask]
        y_sub = y_data.loc[mask]

        y_pred = model_final.predict(X_sub)
        cm = confusion_matrix(y_sub, y_pred, labels=[0, 1])

        plt.figure(figsize=(6, 4))
        sns.heatmap(
            cm,
            annot=True,
            fmt="d",
            cmap="Greens",
            cbar=False,
            xticklabels=["Kosong (0)", "Sedia (1)"],
            yticklabels=["Kosong (0)", "Sedia (1)"]
        )
        plt.title(f"{judul_prefix}: {grup}", fontsize=13,
fontweight="bold")
        plt.xlabel("Prediksi")
        plt.ylabel("Aktual")
        plt.tight_layout()
        plt.savefig(
            os.path.join(output_dir,
f"{prefix_file}_{safe_name(grup)}.png"),
            dpi=300,
            bbox_inches="tight"
        )
        plt.show()

# =====
# B3R6
# =====

mapping_b3r6 = {
    0: "Tidak ada SK",
    1: "Terpencil/Sangat Terpencil",
    2: "Perdesaan",

```

```

    3: "Perkotaan"
}

kode_b3r6 = meta_test["B3R6"].apply(ambil_kode_angka)
grup_b3r6 = kode_b3r6.map(mapping_b3r6)

# skip 0 tidak ada SK
mask_b3r6_valid = grup_b3r6 != "Tidak ada SK"

X_test_b3r6 = X_test.loc[mask_b3r6_valid].copy()
y_test_b3r6 = y_test_series.loc[mask_b3r6_valid].copy()
grup_b3r6_valid = grup_b3r6.loc[mask_b3r6_valid].copy()

print("\nJumlah data B3R6:")
print(grup_b3r6_valid.value_counts(dropna=False))

df_eval_b3r6 = evaluasi_per_grup(
    model=model_final,
    X_data=X_test_b3r6,
    y_data=y_test_b3r6,
    grup_series=grup_b3r6_valid,
    nama_kolom="Kategori B3R6"
)

print("\n=== Evaluasi RF per B3R6 ===")
print(df_eval_b3r6.round(4).to_string(index=False))

df_eval_b3r6.to_excel(
    os.path.join(output_dir, "Evaluasi_RF_Per_B3R6.xlsx"),
    index=False
)

plot_cm_per_grup(
    X_data=X_test_b3r6,
    y_data=y_test_b3r6,
    grup_series=grup_b3r6_valid,
    prefix_file="CM_RF_B3R6",
    judul_prefix="Confusion Matrix RF Tuned - B3R6"
)

# =====
# JAWA-BALI VS LUAR
# =====

kode_b1r1 = meta_test["B1R1"].apply(ambil_kode_angka)

kode_jawa_bali = [31, 32, 33, 34, 35, 36, 51]

grup_wilayah = kode_b1r1.apply(
    lambda x: "Jawa-Bali" if x in kode_jawa_bali else "Luar Jawa-Bali"
)

```

```

)

print("\nJumlah data wilayah:")
print(grup_wilayah.value_counts(dropna=False))

df_eval_wilayah = evaluasi_per_grup(
    model=model_final,
    X_data=X_test,
    y_data=y_test_series,
    grup_series=grup_wilayah,
    nama_kolom="Wilayah"
)

print("\n=== Evaluasi RF Jawa-Bali vs Luar Jawa-Bali ===")
print(df_eval_wilayah.round(4).to_string(index=False))

df_eval_wilayah.to_excel(
    os.path.join(output_dir, "Evaluasi_RF_JawaBali_vs_Luar.xlsx"),
    index=False
)

plot_cm_per_grup(
    X_data=X_test,
    y_data=y_test_series,
    grup_series=grup_wilayah,
    prefix_file="CM_RF_Wilayah",
    judul_prefix="Confusion Matrix RF Tuned - Wilayah"
)

print("\n Metrik dan confusion matrix selesai.")
print("File tersimpan di:", output_dir)

Cek df_raw ada? True
Index X_test contoh: [1058, 2998, 4195, 1372, 4542, 6149, 7763, 4143,
4789, 3785]
Index df_raw range: 0 sampai 9830

meta_test berhasil dibuat dari index X_test.

```

	B1R1	B3R6
1058	13	1.0
2998	32	2.0
4195	33	1.0
1372	14	3.0
4542	33	2.0

```

Missing metadata:
B1R1      0
B3R6     238
dtype: int64

```

Jumlah data B3R6:

B3R6

Perdesaan 812

Terpencil/Sangat Terpencil 492

Perkotaan 283

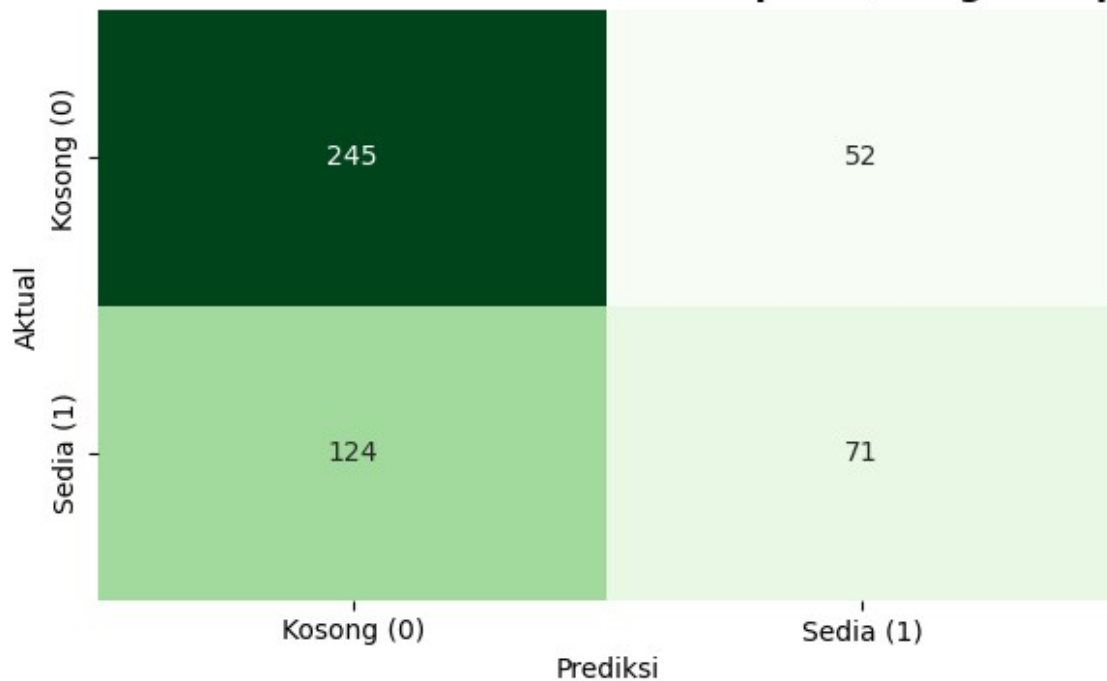
NaN 238

Name: count, dtype: int64

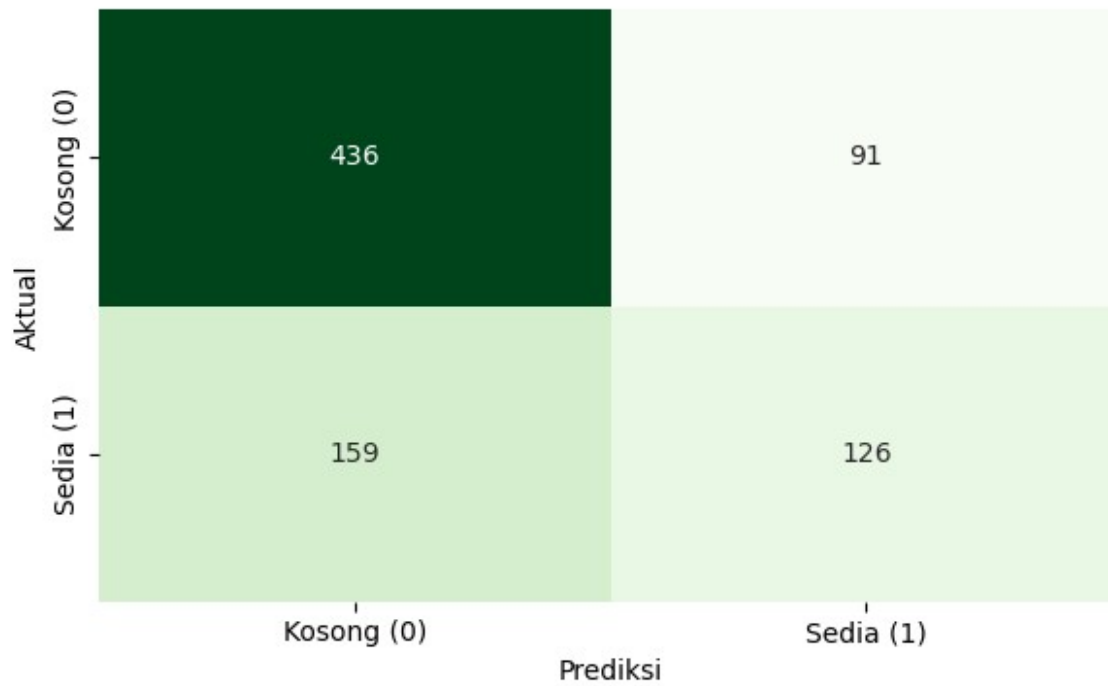
=== Evaluasi RF per B3R6 ===

		Kategori B3R6		Jumlah Data		Kelas 0		Kelas 1	Accuracy
F1 Score	MCC	AUCPRC	AUROC	TN	FP	FN	TP		
Terpencil/Sangat Terpencil				492		297	195	0.6423	
0.4465	0.2135	0.5866	0.6788	245	52	124	71		
Perdesaan				812		527	285	0.6921	
0.5020	0.2906	0.5618	0.7233	436	91	159	126		
Perkotaan				283		188	95	0.6855	
0.2764	0.1905	0.5039	0.6965	177	11	78	17		

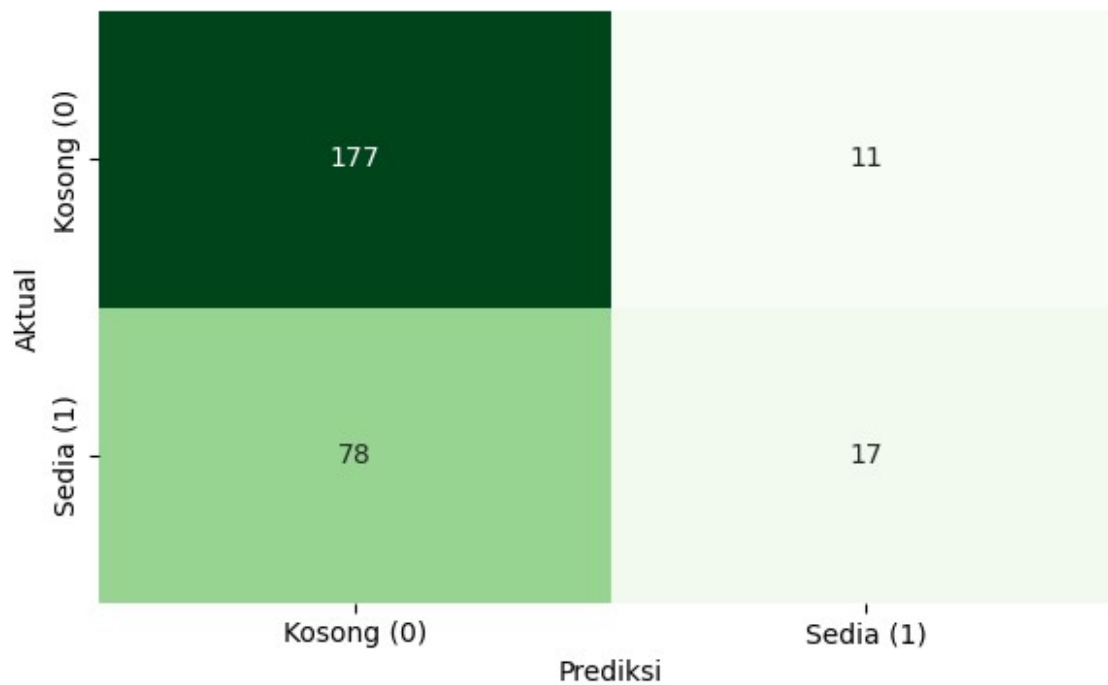
Confusion Matrix RF Tuned - B3R6: Terpencil/Sangat Terpencil



Confusion Matrix RF Tuned - B3R6: Perdesaan



Confusion Matrix RF Tuned - B3R6: Perkotaan

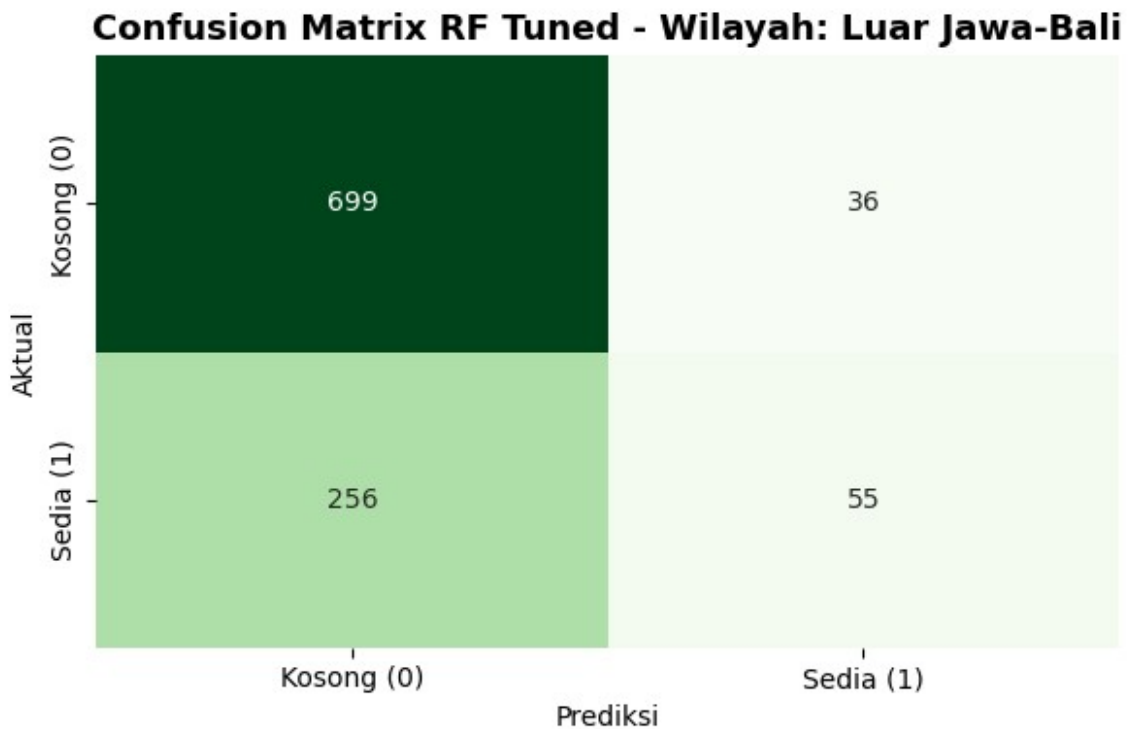


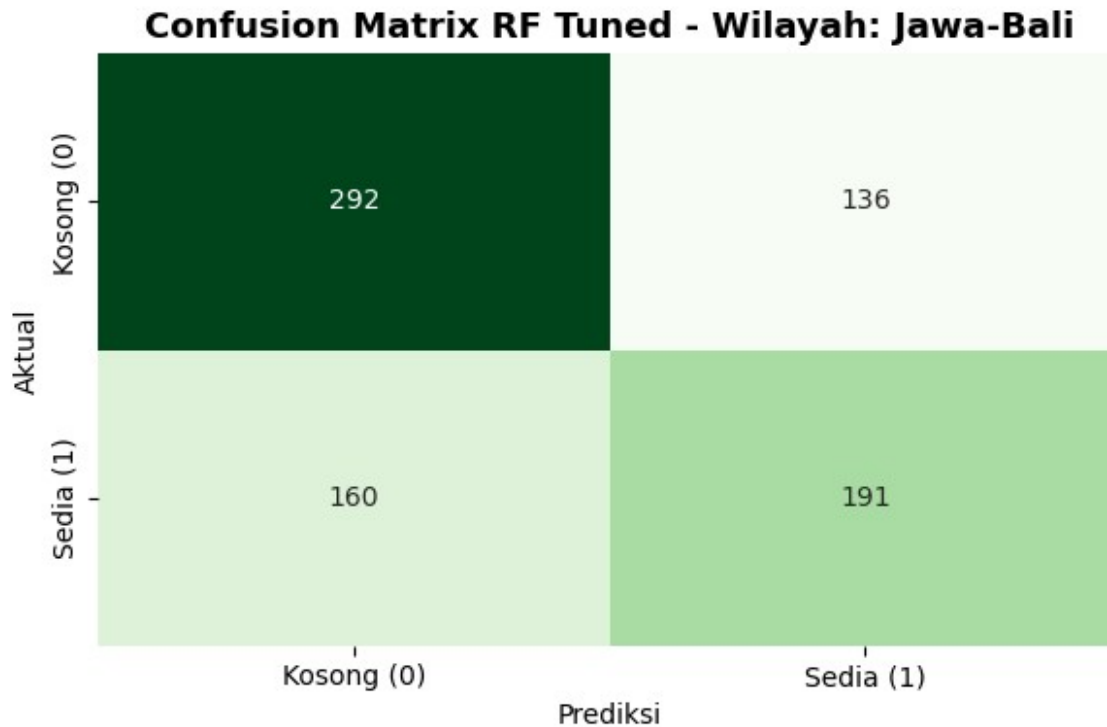
Jumlah data wilayah:
B1R1
Luar Jawa-Bali 1046


```
Jawa-Bali          779
Name: count, dtype: int64
```

```
=== Evaluasi RF Jawa-Bali vs Luar Jawa-Bali ===
```

	Wilayah	Jumlah Data	Kelas 0	Kelas 1	Accuracy	F1 Score	
MCC	AUCPRC	AUROC	TN	FP	FN	TP	
Luar Jawa-Bali			1046	735	311	0.7208	0.2736
0.2074	0.4810	0.7040	699	36	256	55	
Jawa-Bali			779	428	351	0.6200	0.5634
0.2283	0.6243	0.6718	292	136	160	191	





```

□ Metrik dan confusion matrix selesai.
File tersimpan di: /Users/gitajilanmahira/Downloads/Gita
Jilan_Rifaskes 2/Analisis_Model_Final_RF

# =====
# NEXT: SHAP IMPORTANCE + BEESWARM PER B3R6 DAN WILAYAH
# Run setelah global SHAP dan meta_test berhasil dibuat
# =====

import os
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import shap

folder_path = "/Users/gitajilanmahira/Downloads/Gita Jilan_Rifaskes 2"
output_dir = os.path.join(folder_path, "Analisis_Model_Final_RF")
os.makedirs(output_dir, exist_ok=True)

# =====
# 1. CEK VARIABEL PENTING
# =====

if "X_test_shap" not in globals():
    raise ValueError("X_test_shap belum ada. Run global SHAP dulu.")

```

```

if "shap_values_class1" not in globals():
    raise ValueError("shap_values_class1 belum ada. Run global SHAP
dulu.")

if "meta_test" not in globals():
    raise ValueError("meta_test belum ada. Run cell metrik/confusion
matrix sebelumnya dulu.")

print("X_test_shap:", X_test_shap.shape)
print("shap_values_class1:", np.array(shap_values_class1).shape)
print("meta_test:", meta_test.shape)

# =====
# 2. FUNGSI BANTU
# =====

def ambil_kode_angka(x):
    if pd.isna(x):
        return np.nan
    teks = str(x).strip()
    angka = re.findall(r"\d+", teks)
    if len(angka) == 0:
        return np.nan
    return int(angka[0])

def safe_name(x):
    return str(x).replace("/", "_").replace(" ", "_").replace("-",
"_").replace(":", "")

# Buat dataframe SHAP
df_shap_values = pd.DataFrame(
    shap_values_class1,
    columns=X_test_shap.columns,
    index=X_test_shap.index
)

print("\n Data SHAP siap:", df_shap_values.shape)

# =====
# 3. BUAT GRUP B3R6 DAN WILAYAH DARI meta_test
# =====

mapping_b3r6 = {
    0: "Tidak ada SK",
    1: "Terpencil/Sangat Terpencil",
    2: "Perdesaan",
    3: "Perkotaan"
}

```

```

kode_b3r6 = meta_test["B3R6"].apply(ambil_kode_angka)
grup_b3r6 = kode_b3r6.map(mapping_b3r6)

# Skip logic 0
grup_b3r6 = grup_b3r6.mask(grup_b3r6 == "Tidak ada SK")

kode_b1r1 = meta_test["B1R1"].apply(ambil_kode_angka)

kode_jawa_bali = [31, 32, 33, 34, 35, 36, 51]

grup_wilayah = kode_b1r1.apply(
    lambda x: "Jawa-Bali" if x in kode_jawa_bali else "Luar Jawa-Bali"
)

print("\nJumlah B3R6 pada test set:")
print(grup_b3r6.value_counts(dropna=False))

print("\nJumlah wilayah pada test set:")
print(grup_wilayah.value_counts(dropna=False))

# =====
# 4. FUNGSI SHAP PER GRUP
# =====

def shap_per_grup_dari_global(grup_series, nama_kolom, prefix_file,
top_n=15, buat_beeswarm=True):
    """
    Menggunakan SHAP global yang sudah dihitung.
    Aman untuk SHAP full atau sample karena pakai reindex.
    """

    # Sesuaikan grup ke index SHAP
    grup_shap = grup_series.reindex(df_shap_values.index)

    hasil = []

    for grup in grup_shap.dropna().unique():
        idx = grup_shap[grup_shap == grup].index

        if len(idx) < 5:
            print(f"\nSkip {grup}, data SHAP terlalu sedikit:
{len(idx)}")
            continue

        shap_sub = df_shap_values.loc[idx]
        X_sub_shap = X_test_shap.loc[idx]

        mean_abs_shap = np.abs(shap_sub).mean(axis=0)

        df_temp = pd.DataFrame({

```

```

        nama_kolom: grup,
        "Fitur": mean_abs_shap.index,
        "Mean_Abs_SHAP": mean_abs_shap.values
    }).sort_values(by="Mean_Abs_SHAP", ascending=False)

    hasil.append(df_temp.head(20))

    print(f"\n===== Top {top_n} SHAP Importance - {grup} =====")
    print(df_temp.head(top_n).to_string(index=False))

    safe_grup = safe_name(grup)

    # Bar plot Mean Abs SHAP
    df_plot = df_temp.head(top_n).iloc[::-1]

    plt.figure(figsize=(10, 7))
    plt.barh(df_plot["Fitur"], df_plot["Mean_Abs_SHAP"])
    plt.title(f"Top {top_n} SHAP Feature Importance - {grup}",
    fontsize=13, fontweight="bold")
    plt.xlabel("Mean Absolute SHAP")
    plt.ylabel("Fitur")
    plt.tight_layout()
    plt.savefig(
        os.path.join(output_dir,
    f"{prefix_file}_Bar_{safe_grup}.png"),
        dpi=300,
        bbox_inches="tight"
    )
    plt.show()

    # Beeswarm per grup
    if buat_beeswarm:
        shap.summary_plot(
            shap_sub.values,
            X_sub_shap,
            max_display=top_n,
            show=False
        )

        plt.title(f"SHAP Beeswarm RF Tuned - {grup}", fontsize=13,
    fontweight="bold")
        plt.tight_layout()
        plt.savefig(
            os.path.join(output_dir,
    f"{prefix_file}_Beeswarm_{safe_grup}.png"),
            dpi=300,
            bbox_inches="tight"
        )
        plt.show()

```

```

    if len(hasil) == 0:
        return pd.DataFrame()

    df_hasil = pd.concat(hasil, ignore_index=True)
    df_hasil.to_excel(
        os.path.join(output_dir, f"{prefix_file}_Table.xlsx"),
        index=False
    )

    return df_hasil

# =====
# 5. RUN B3R6
# =====

df_shap_b3r6 = shap_per_grup_dari_global(
    grup_series=grup_b3r6,
    nama_kolom="Kategori B3R6",
    prefix_file="SHAP_RF_B3R6",
    top_n=15,
    buat_beeswarm=True
)

# =====
# 6. RUN JAWA-BALI VS LUAR
# =====

df_shap_wilayah = shap_per_grup_dari_global(
    grup_series=grup_wilayah,
    nama_kolom="Wilayah",
    prefix_file="SHAP_RF_Wilayah",
    top_n=15,
    buat_beeswarm=True
)

print("\n SHAP per kategori selesai.")
print("File tersimpan di:", output_dir)

X_test_shap: (1825, 89)
shap_values_class1: (1825, 89)
meta_test: (1825, 2)

Data SHAP siap: (1825, 89)

Jumlah B3R6 pada test set:
B3R6
Perdesaan 812
Terpencil/Sangat Terpencil 492
Perkotaan 283

```

NaN 238

Name: count, dtype: int64

Jumlah wilayah pada test set:

B1R1

Luar Jawa-Bali 1046

Jawa-Bali 779

Name: count, dtype: int64

===== Top 15 SHAP Importance - Terpencil/Sangat Terpencil =====
Kategori B3R6 Fitur

Mean_Abs_SHAP

Terpencil/Sangat Terpencil jalur_num_B9R13A
0.027491

Terpencil/Sangat Terpencil remainder__X_Ketersediaan_Dinkes
0.023754

Terpencil/Sangat Terpencil jalur_num_B4R16A
0.022165

Terpencil/Sangat Terpencil remainder__X_Sedia_Obat_NonFornas
0.020739

Terpencil/Sangat Terpencil jalur_num_B4R14
0.017679

Terpencil/Sangat Terpencil remainder_B3R9B
0.017221

Terpencil/Sangat Terpencil remainder_B3R3
0.015561

Terpencil/Sangat Terpencil jalur_num_B3R11E
0.011467

Terpencil/Sangat Terpencil remainder_B3R10
0.010957

Terpencil/Sangat Terpencil jalur_nom__X_Sumber_Dana_BPJS_1
0.009093

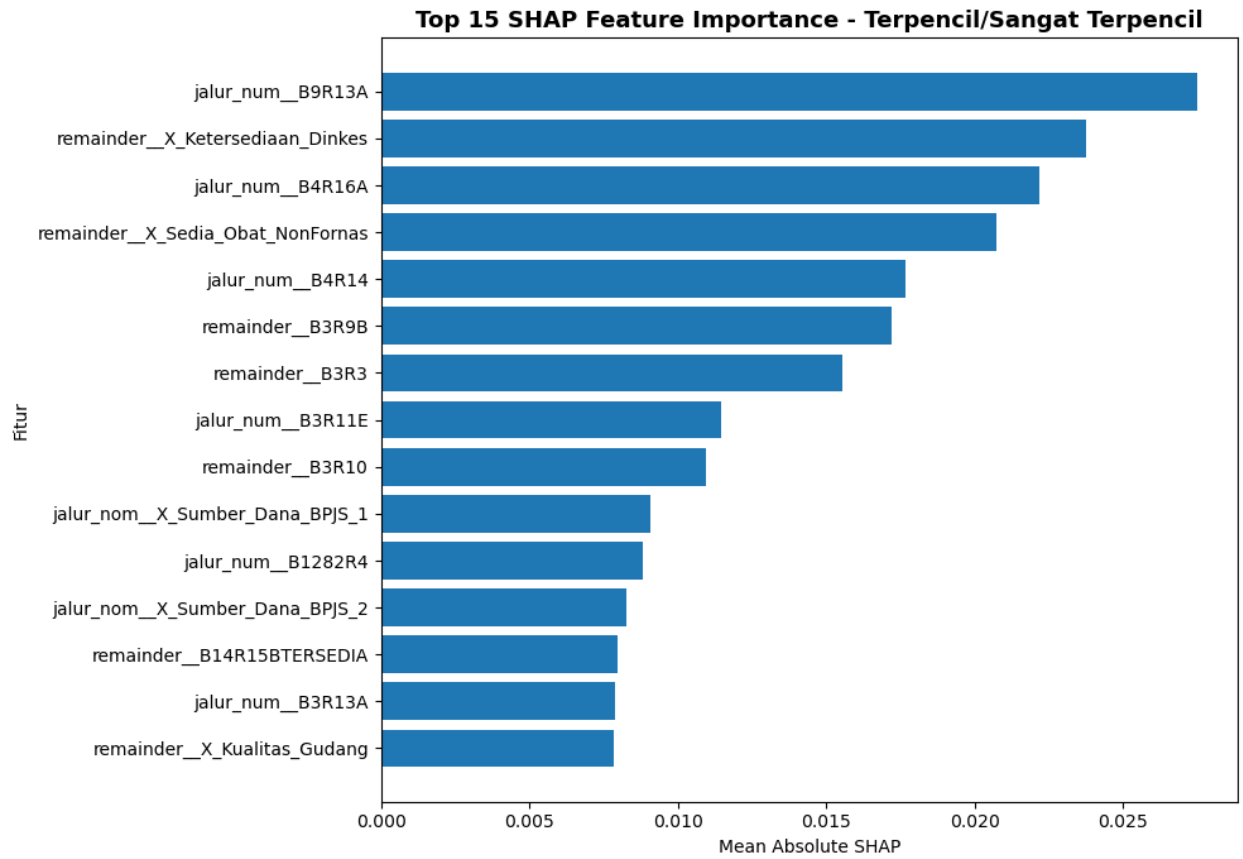
Terpencil/Sangat Terpencil jalur_num_B1282R4
0.008828

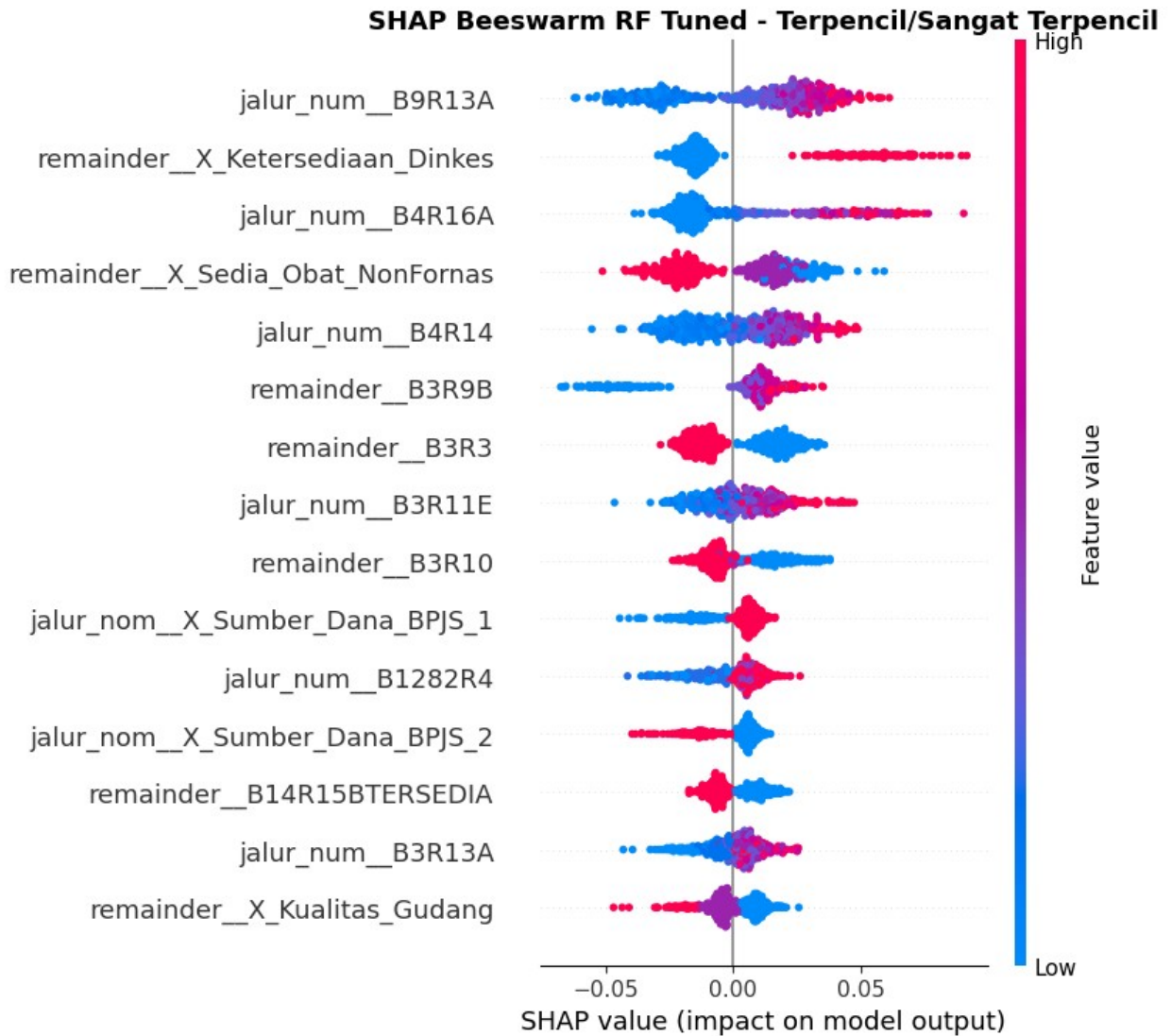
Terpencil/Sangat Terpencil jalur_nom__X_Sumber_Dana_BPJS_2
0.008287

Terpencil/Sangat Terpencil remainder_B14R15BTERSEDIA
0.007959

Terpencil/Sangat Terpencil jalur_num_B3R13A
0.007862

Terpencil/Sangat Terpencil remainder__X_Kualitas_Gudang
0.007838

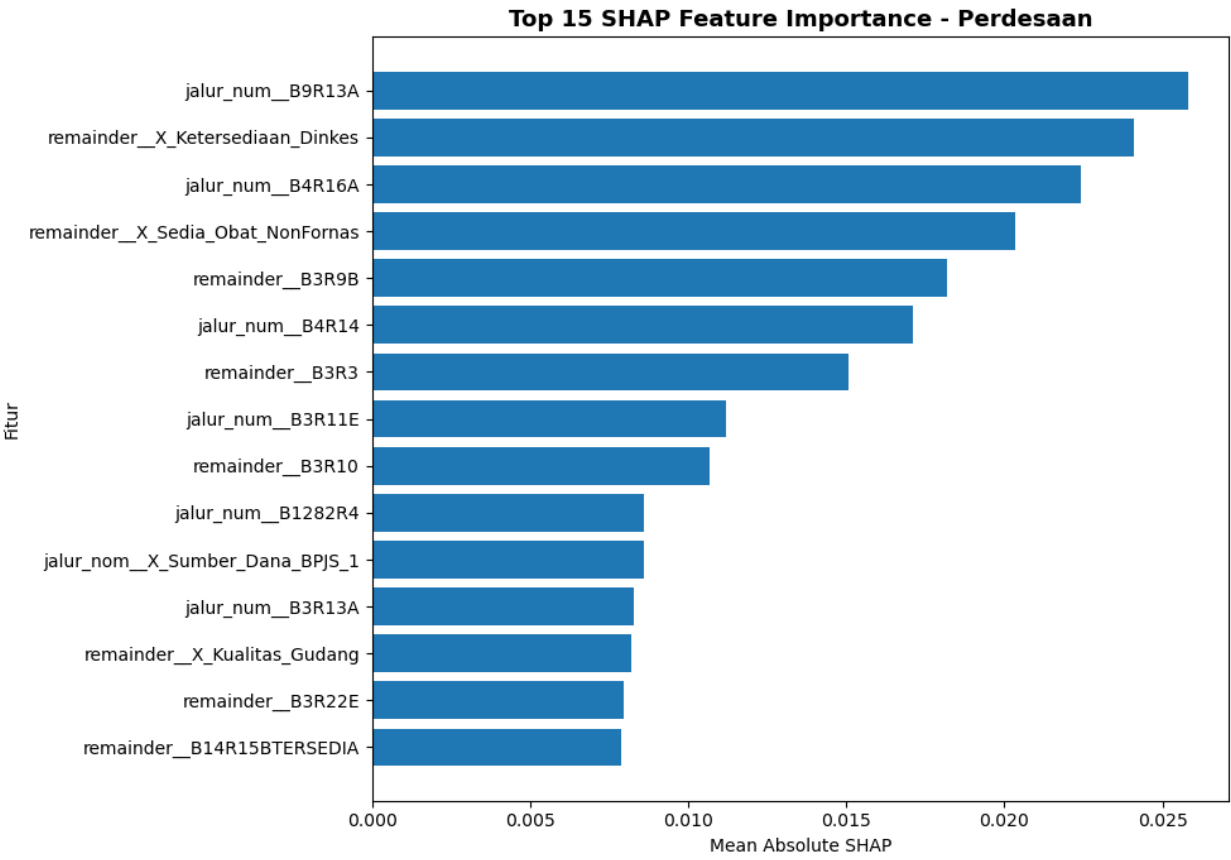


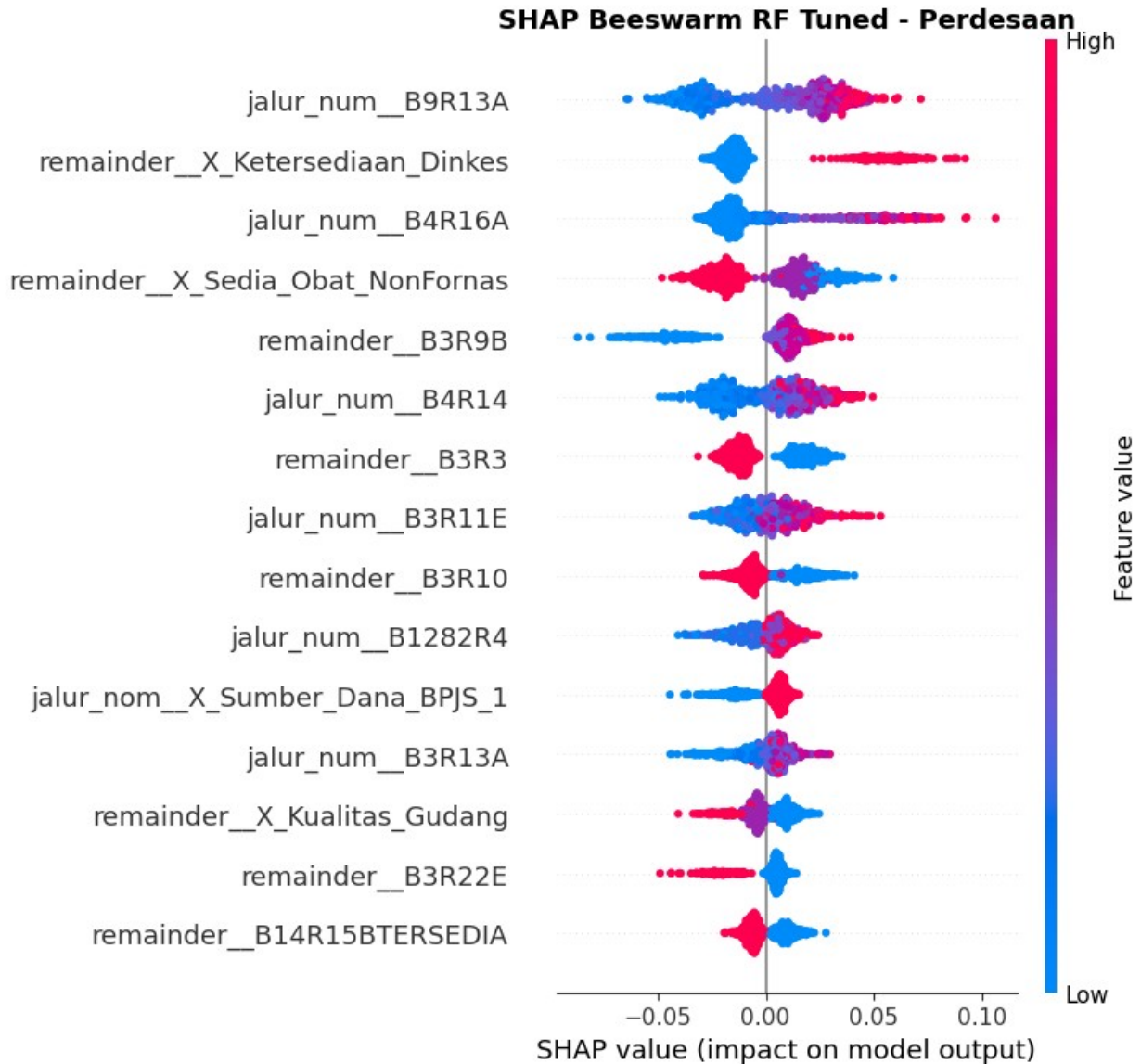


===== Top 15 SHAP Importance - Perdesaan =====

Kategori B3R6	Fitur	Mean_Abs_SHAP
Perdesaan	jalur_num_B9R13A	0.025804
Perdesaan	remainder_X_Ketersediaan_Dinkes	0.024113
Perdesaan	jalur_num_B4R16A	0.022432
Perdesaan	remainder_X_Sedia_Obat_NonFornas	0.020329
Perdesaan	remainder_B3R9B	0.018201
Perdesaan	jalur_num_B4R14	0.017119
Perdesaan	remainder_B3R3	0.015085
Perdesaan	jalur_num_B3R11E	0.011198
Perdesaan	remainder_B3R10	0.010675
Perdesaan	jalur_num_B1282R4	0.008618
Perdesaan	jalur_nom_X_Sumber_Dana_BPJS_1	0.008580
Perdesaan	jalur_num_B3R13A	0.008271
Perdesaan	remainder_X_Kualitas_Gudang	0.008210

Perdesaan	remainder__B3R22E	0.007978
Perdesaan	remainder__B14R15BTERSEDIA	0.007861

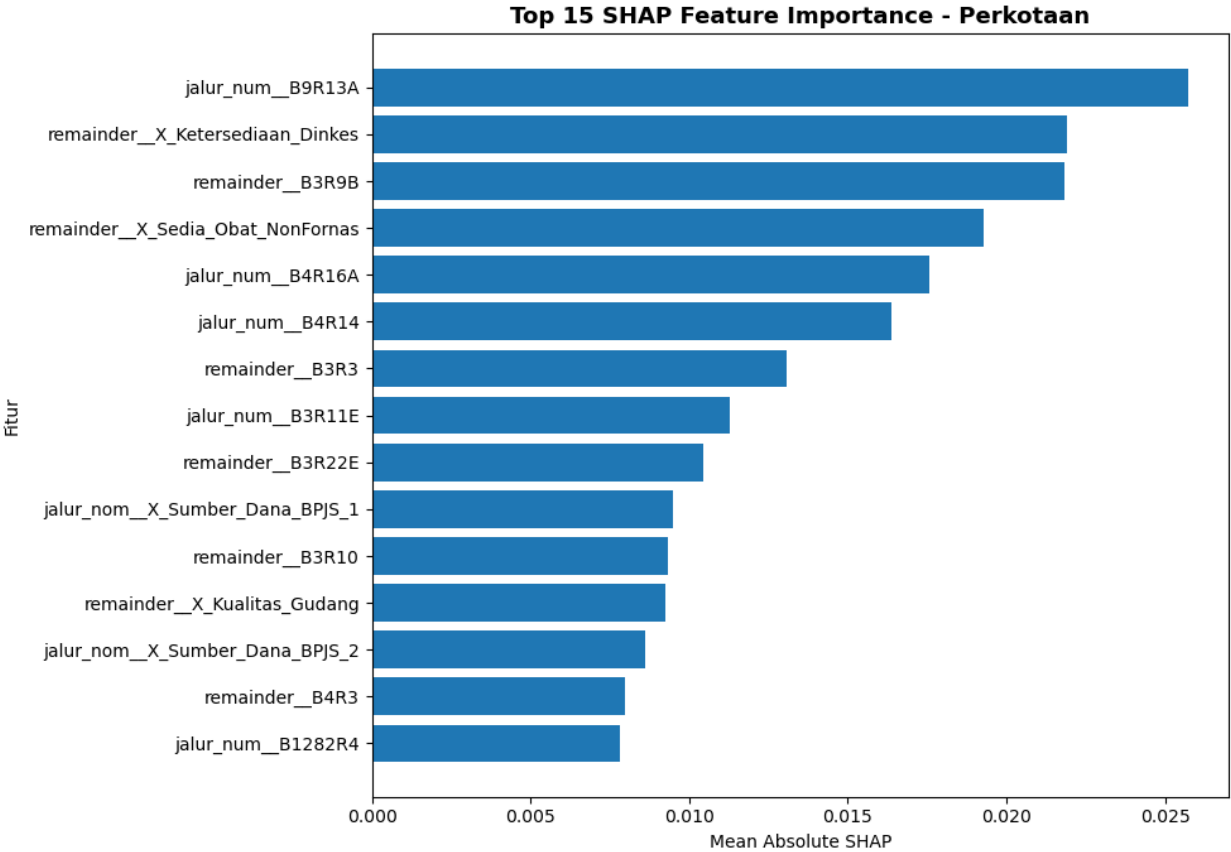


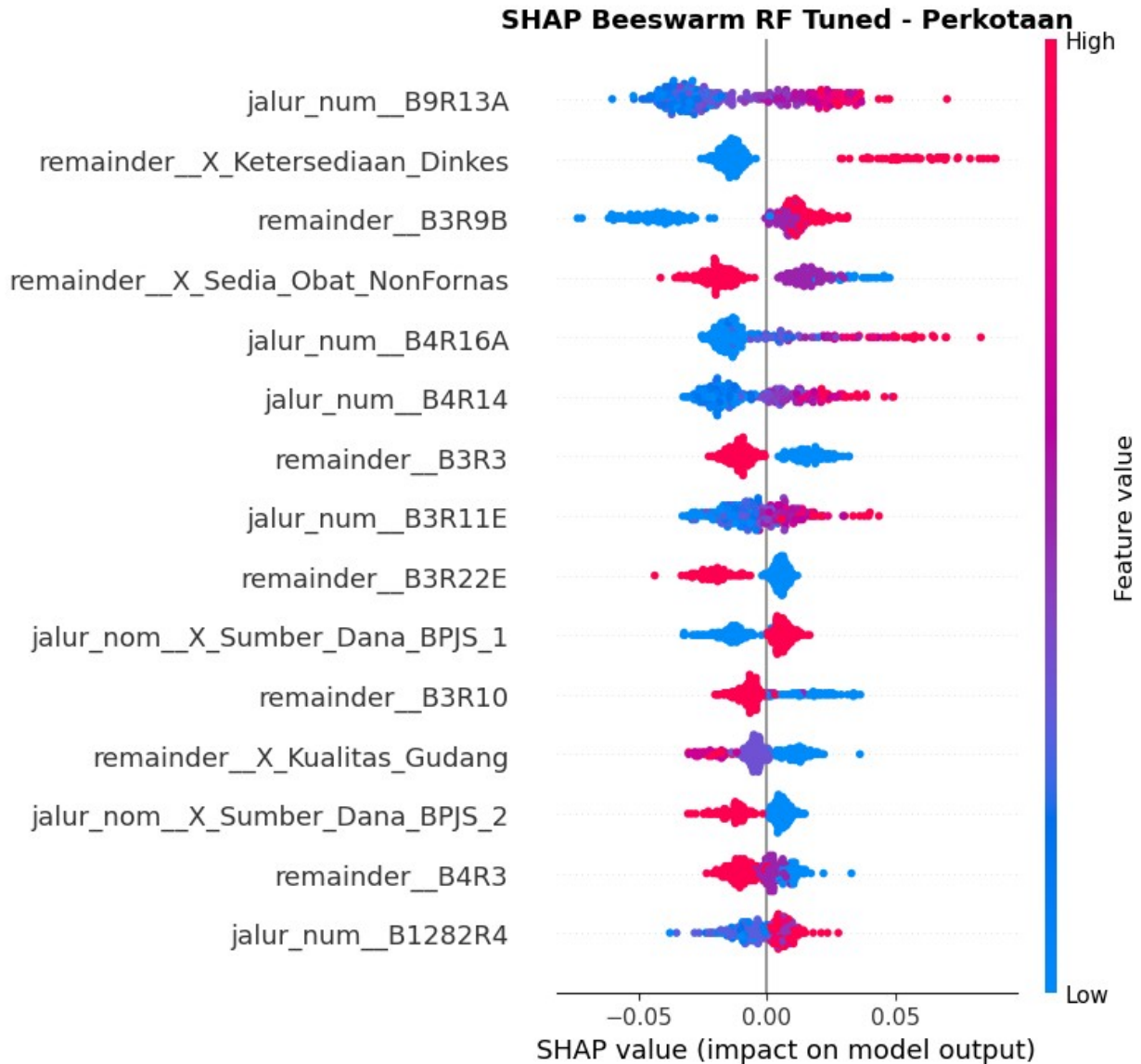


===== Top 15 SHAP Importance - Perkotaan =====

Kategori B3R6	Fitur	Mean_Abs_SHAP
Perkotaan	jalur_num_B9R13A	0.025721
Perkotaan	remainder_X_Ketersediaan_Dinkes	0.021923
Perkotaan	remainder_B3R9B	0.021815
Perkotaan	remainder_X_Sedia_Obat_NonFornas	0.019278
Perkotaan	jalur_num_B4R16A	0.017577
Perkotaan	jalur_num_B4R14	0.016362
Perkotaan	remainder_B3R3	0.013060
Perkotaan	jalur_num_B3R11E	0.011280
Perkotaan	remainder_B3R22E	0.010434
Perkotaan	jalur_nom_X_Sumber_Dana_BPJS_1	0.009505
Perkotaan	remainder_B3R10	0.009309

Perkotaan	remainder__X_Kualitas_Gudang	0.009260
Perkotaan	jalur_nom__X_Sumber_Dana_BPJS_2	0.008592
Perkotaan	remainder__B4R3	0.007955
Perkotaan	jalur_num__B1282R4	0.007828

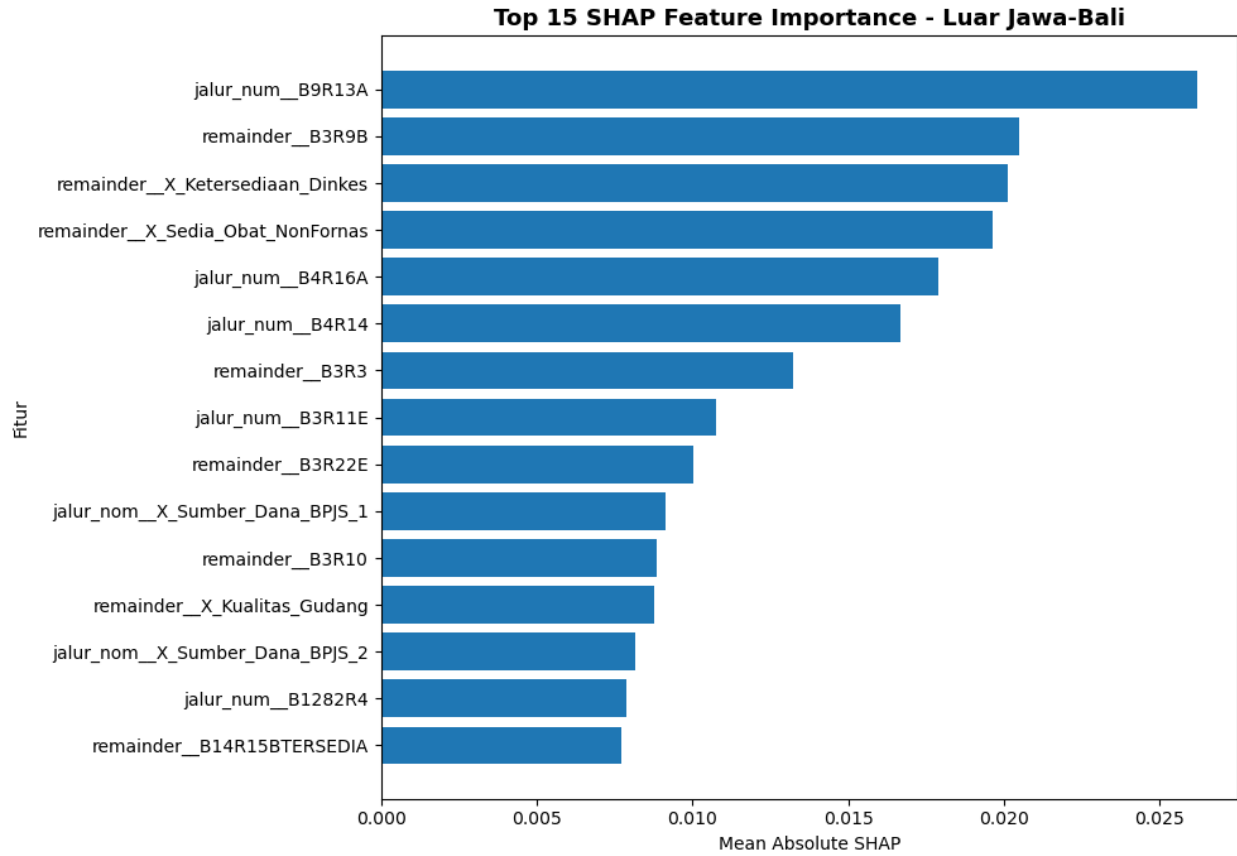


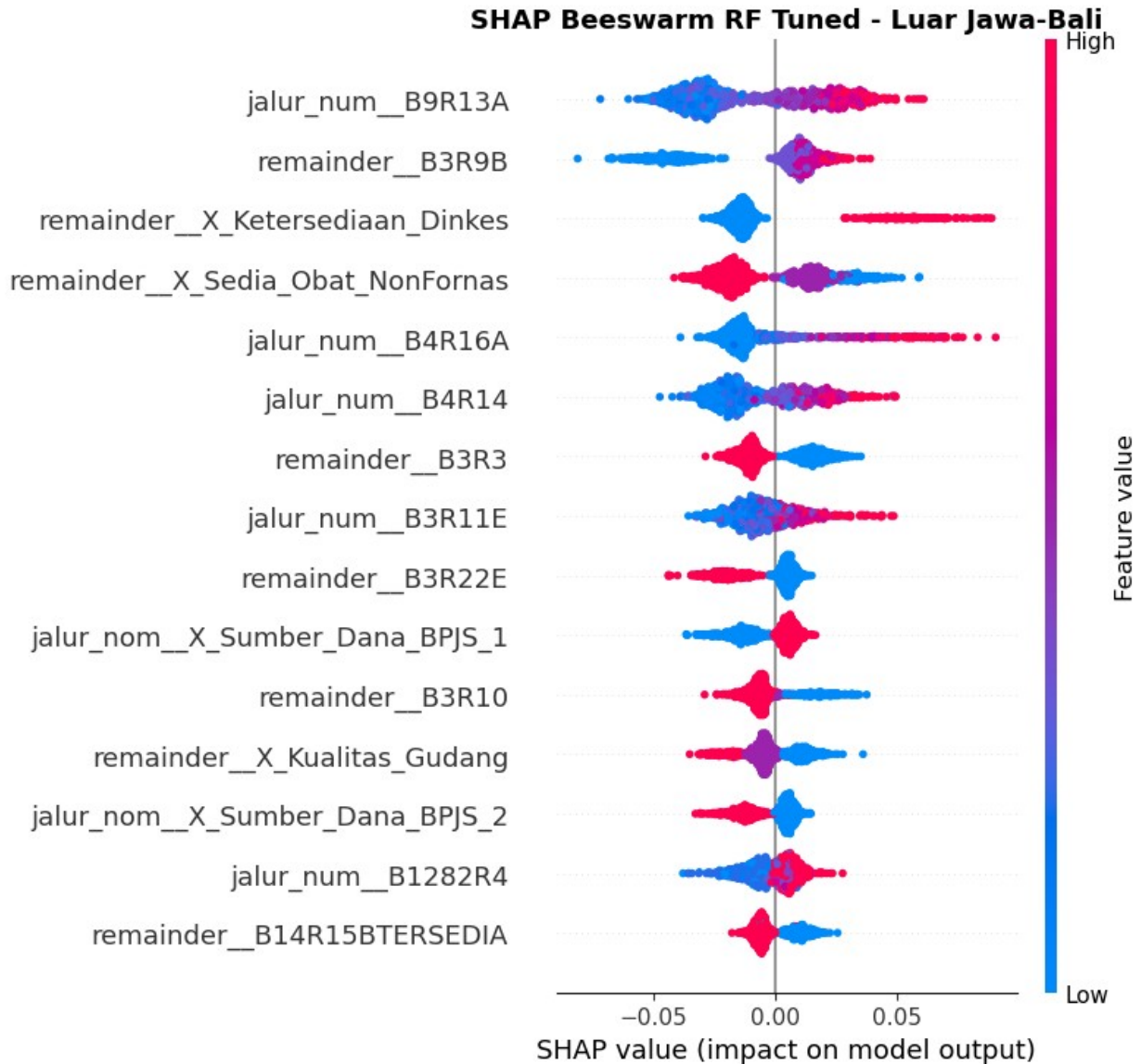


===== Top 15 SHAP Importance - Luar Jawa-Bali =====

Wilayah	Fitur	Mean_Abs_SHAP
Luar Jawa-Bali	jalur_num_B9R13A	0.026199
Luar Jawa-Bali	remainder_B3R9B	0.020492
Luar Jawa-Bali	remainder_X_Ketersediaan_Dinkes	0.020143
Luar Jawa-Bali	remainder_X_Sedia_Obat_NonFornas	0.019634
Luar Jawa-Bali	jalur_num_B4R16A	0.017877
Luar Jawa-Bali	jalur_num_B4R14	0.016676
Luar Jawa-Bali	remainder_B3R3	0.013218
Luar Jawa-Bali	jalur_num_B3R11E	0.010754
Luar Jawa-Bali	remainder_B3R22E	0.010024
Luar Jawa-Bali	jalur_nom_X_Sumber_Dana_BPJS_1	0.009120
Luar Jawa-Bali	remainder_B3R10	0.008865

Luar Jawa-Bali	remainder__X_Kualitas_Gudang	0.008780
Luar Jawa-Bali	jalur_nom__X_Sumber_Dana_BPJS_2	0.008179
Luar Jawa-Bali	jalur_num__B1282R4	0.007873
Luar Jawa-Bali	remainder__B14R15BTERSEDIA	0.007720

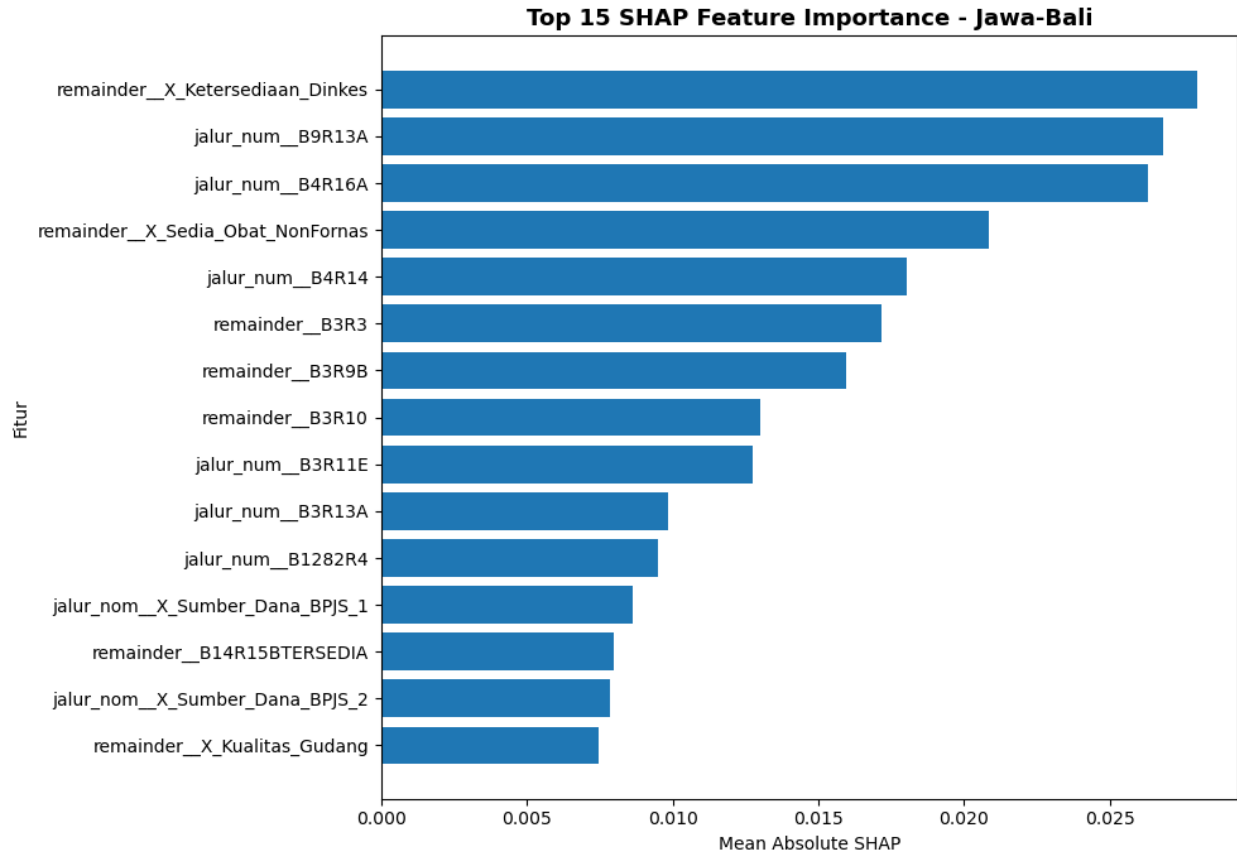


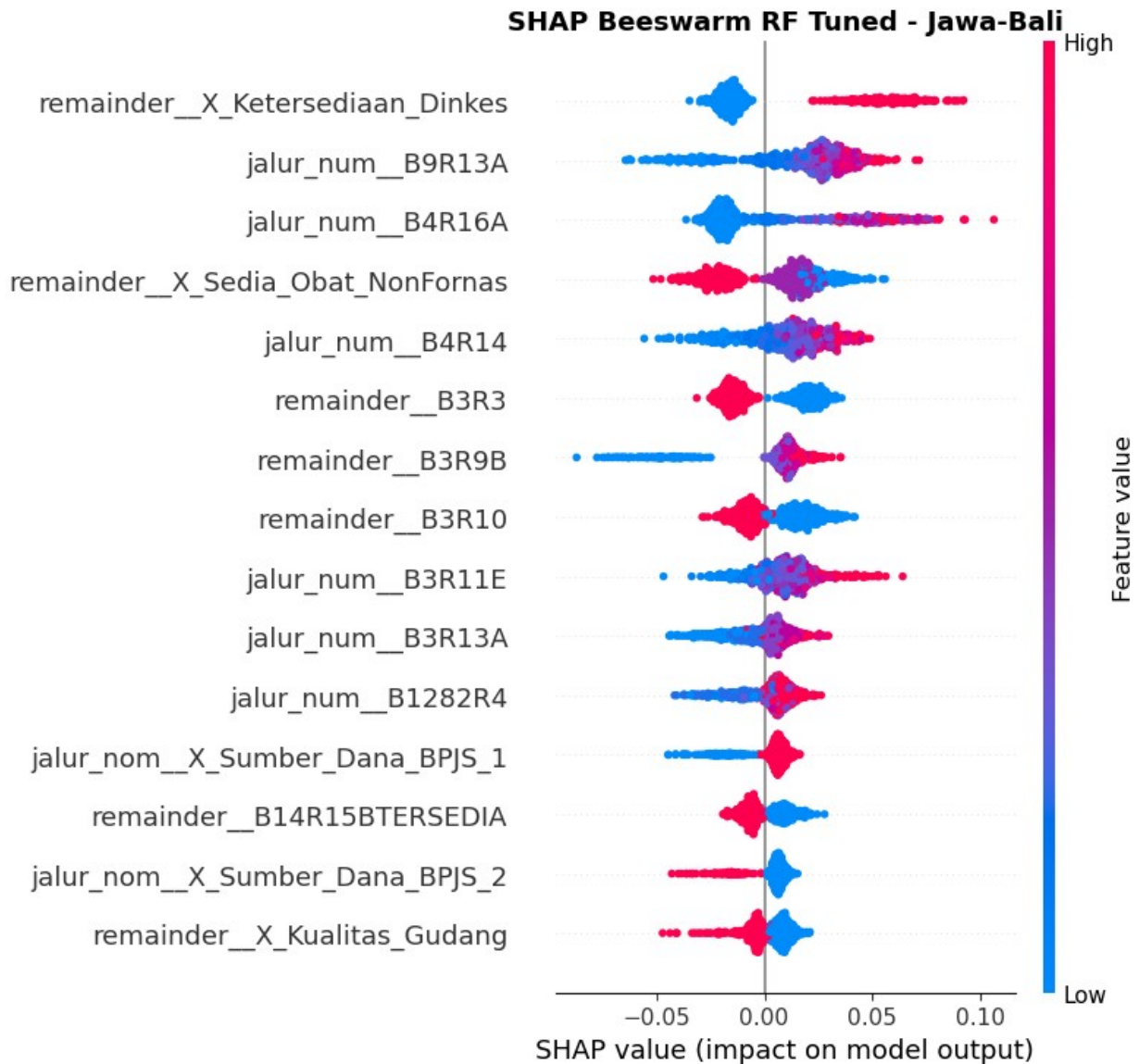


===== Top 15 SHAP Importance - Jawa-Bali =====

Wilayah	Fitur	Mean_Abs_SHAP
Jawa-Bali	remainder_X_Ketersediaan_Dinkes	0.027982
Jawa-Bali	jalur_num_B9R13A	0.026834
Jawa-Bali	jalur_num_B4R16A	0.026297
Jawa-Bali	remainder_X_Sedia_Obat_NonFornas	0.020864
Jawa-Bali	jalur_num_B4R14	0.018035
Jawa-Bali	remainder_B3R3	0.017185
Jawa-Bali	remainder_B3R9B	0.015955
Jawa-Bali	remainder_B3R10	0.013000
Jawa-Bali	jalur_num_B3R11E	0.012737
Jawa-Bali	jalur_num_B3R13A	0.009828
Jawa-Bali	jalur_num_B1282R4	0.009487

Jawa-Bali	jalur_nom__X_Sumber_Dana_BPJS_1	0.008625
Jawa-Bali	remainder__B14R15BTERSEDIA	0.007983
Jawa-Bali	jalur_nom__X_Sumber_Dana_BPJS_2	0.007870
Jawa-Bali	remainder__X_Kualitas_Gudang	0.007470





□ SHAP per kategori selesai.

File tersimpan di: /Users/gitajilanmahira/Downloads/Gita
Jilan_Rifaskes 2/Analisis_Model_Final_RF